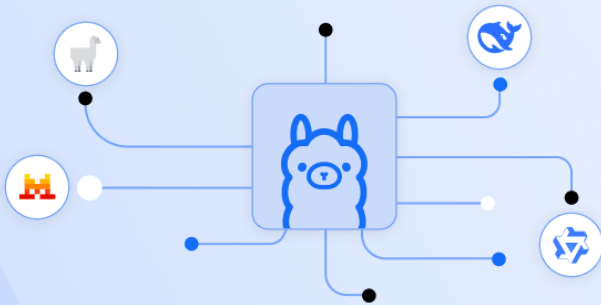


Ollama Models List: Top AI Models
You Can Run Locally



OLLAMA

Run Real LLMs on Your Own Laptop

No API key, no internet required, no cost
per token just `ollama run`



AI Coach John

FOLLOW



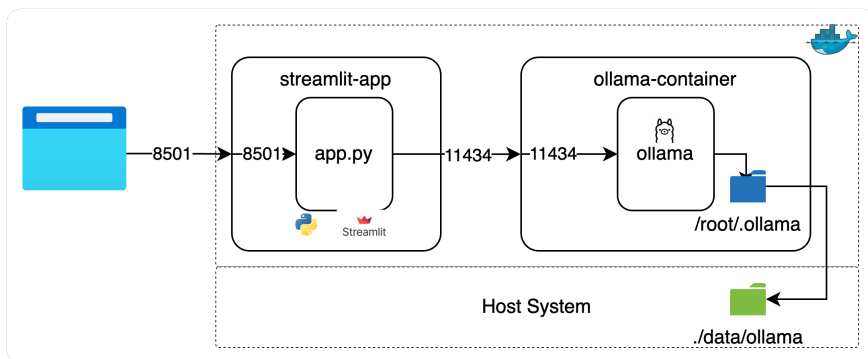
PROITBRIDGE
Drive For Better Future

Repost

? WHAT IS OLLAMA?

A local runtime for open-weight LLMs

Ollama is a free tool that downloads, manages, and runs open-source LLMs (Llama, Qwen, Gemma, DeepSeek, and hundreds more) directly on your own machine no cloud API, no internet connection required after the model is downloaded.



≡ UNDER THE HOOD

It installs a lightweight background service that loads a model into memory and exposes it through a local REST API and a simple CLI so any app you build talks to `localhost` instead of `api.openai.com`.



AI Coach John

FOLLOW



PROTIBRIDGE
Drive For Better Future

↺ Repost

Why students & engineers **actually** use this

Zero cost per token

Once downloaded, there's no usage-based billing; run as many requests as your hardware can handle.

Full privacy

Your prompts and data never leave your machine, ideal for sensitive data or personal projects.

Works offline

No internet dependency after the initial model download.

Full control

Swap models instantly, tune parameters, and customize behavior via Modelfiles.

Great for learning

Experiment with real models without worrying about API costs while you learn.



AI Coach John

FOLLOW



PROTONBRIDGE
Drive For Better Future

 **Repost**

What's happening when **you run a model**

```
ollama pull llama3.1    # downloads a quantized model (GGUF)
ollama run llama3.1    # loads into memory, starts chat
                        # session
                        # Ollama also runs a background API
server                  # at http://localhost:11434
```

THE MAGIC STEP

The installer sets up a background service (systemd on Linux, menu-bar app on Mac) that keeps a local API listening this is what lets other applications (your Python scripts, LangChain, Open WebUI) send requests to your local models exactly like they would to a cloud API.



AI Coach John

FOLLOW



PROITBRIDGE
Drive For Better Future

 Repost

Getting set up, **step by step**

```
# macOS / Linux
curl -fsSL https://ollama.com/install.sh | sh

# Windows / macOS
# Download the installer directly from ollama.com/download

# Verify the install
ollama --version

# Pull your first model
ollama pull llama3.2

# Start chatting right in your terminal
ollama run llama3.2
```

HARDWARE AUTO-DETECT

The installer automatically detects your GPU (NVIDIA/AMD/Apple Silicon) and configures itself to use it if available falling back to CPU-only mode if not. After install, the API is live at `localhost:11434`.



AI Coach John

FOLLOW



PROITBRIDGE
Drive For Better Future

 **Repost**

The commands you'll use constantly

```
ollama pull <model>      # download model without running it
ollama run <model>        # download (if needed) + start chat
ollama list               # show all locally downloaded
                           models
ollama show <model>       # show params, context,
                           capabilities
ollama ps                 # show currently running/loading
                           models
ollama rm <model>         # delete a model to free disk space
```

🔍 PRO TIP: CHECK BEFORE BUILDING

`ollama show` is especially useful before choosing a model it tells you the context window, quantization, and capabilities like vision, tool-calling, or "thinking" mode, right from your terminal.



AI Coach John

FOLLOW



PROTIBRIDGE
Drive For Better Future

Repost

What kinds of models **are actually** available

The Ollama library hosts several hundred curated open-weight models, spanning:



General chat / RAG models

Llama, Qwen, Gemma family



Coding-specialized models

Qwen3-Coder, Devstral, DeepCoder



Reasoning models

DeepSeek-R1 and its distilled variants



Multimodal (vision/audio)

Gemma 4, LLaVA



Embedding models

omic-embed-text, for RAG pipelines



Efficient open-weight models

gpt-oss, Phi-4

Browse the catalog at ollama.com/library or pull directly.



AI Coach John

FOLLOW



PROTIBRIDGE
Drive For Better Future



Repost

Llama 3.3 70B:

The general-purpose workhorse

	MODEL	llama3.3:70b
	PUBLISHER	Meta
	PARAMETERS	70 billion
	DOWNLOAD SIZE	~43 GB (default quantization)
	CONTEXT	128K tokens
	BEST FOR	General chat, RAG, broad knowledge
	HARDWARE	~40GB+ VRAM recommended

WHY IT MATTERS

This is the model most people reach for when they want strong, well-rounded quality for general assistant tasks or as the generation step in a RAG pipeline the tradeoff is it needs serious hardware (a high-VRAM GPU or a Mac with unified memory) to run at a comfortable speed.



AI Coach John

FOLLOW



PROITBRIDGE
Drive For Better Future

 Repost

Qwen3-Coder 30B:

The best local coding model

	MODEL	qwen3-coder:30b
	PUBLISHER	Alibaba (Qwen team)
	ARCHITECTURE	MoE 30B total, ~3.3B active params
	DOWNLOAD SIZE	~19 GB (Q4_K_M quantization)
	CONTEXT	256K tokens
	BEST FOR	Coding, agentic workflows, long context
	HARDWARE	Runs well on 24-32GB GPU or 32GB Mac

WHY IT MATTERS

Its Mixture-of-Experts design means only a small fraction of its total parameters activate per token so it delivers much stronger coding performance per GB of VRAM than a same-sized "dense" model, making it highly practical.



AI Coach John

FOLLOW



PROITBRIDGE
Drive For Better Future

 Repost

DeepSeek-R1 (Distilled): Bringing "reasoning mode"

 MODEL	deepseek-r1:8b / 14b / 32b
 PUBLISHER	DeepSeek
 STYLE	Distilled reasoning (shows "thinking")
 BEST FOR	Math, logic, multi-step tasks
 HARDWARE	Scales from modest to high-end

WHY IT MATTERS

DeepSeek-R1's reasoning behavior was distilled down into smaller models, so you can get meaningfully better step-by-step reasoning performance at 8B-14B parameters than a similarly-sized non-reasoning model a great pick for logic-heavy tasks on modest hardware.



AI Coach John

FOLLOW



PROITBRIDGE
Drive For Better Future

 Repost

Gemma 4 (Multimodal): Vision, audio & tools

	MODEL	gemma4:12b (also 26b, 31b)
	PUBLISHER	Google
	CAPABILITIES	Vision, audio, tools, thinking mode
	BEST FOR	Multimodal assistants, image RAG

WHY IT MATTERS

Gemma 4 is a meaningful jump over Gemma 3 every size in the family is natively multimodal (not just the largest), and o11ama show gemma4:12b will list vision, audio, tools, and thinking directly under its capabilities, making it a strong choice if your project needs more than plain text.



AI Coach John

FOLLOW



PROITBRIDGE
Drive For Better Future

 Repost

gpt-oss:20b:

OpenAI's open-weight model

	MODEL	<code>gpt-oss:20b</code>
	PUBLISHER	OpenAI
	QUANTIZATION	MXFP4 (native low-precision format)
	DOWNLOAD SIZE	~14 GB
	CONTEXT	128K tokens
	BEST FOR	General tasks on modest hardware
	HARDWARE	Runs on ~16GB of RAM

WHY IT MATTERS

This is one of the most accessible high-quality general models for anyone without a high-end GPU its native low-precision quantization keeps both download size and memory footprint low while remaining genuinely usable for everyday tasks.



AI Coach John

FOLLOW



PROTIBRIDGE
Drive For Better Future

 **Repost**

nomic-embed-text:

The default embedding model

	MODEL	nomic-embed-text
	PUBLISHER	Nomic AI
	TYPE	Text embedding (not a chat model)
	BEST FOR	Powering retrieval step in local RAG

WHY IT MATTERS

One of the most-pulled models in the entire Ollama library this is the model you'd point a local RAG pipeline's embedding step at, letting you build a fully local RAG system (embeddings + vector search + generation) with zero API calls to anyone.



Phi-4:

Compact & high-quality

	MODEL	phi-4:14b
	PUBLISHER	Microsoft
	PARAMETERS	14 billion
	BEST FOR	Strong reasoning relative to small size

WHY IT MATTERS

Phi-4 punches well above its parameter count, making it a great pick when you want solid quality but don't have the hardware (or patience) for a 70B model a good default for students learning on a mid-range laptop.



AI Coach John

FOLLOW



PROITBRIDGE
Drive For Better Future

 Repost

Match the model to **what you actually have**

YOUR SETUP	RECOMMENDED MODEL	WHY
Laptop, no dedicated GPU, ~16GB RAM	<code>gpt-oss:20b / llama3.2:3b</code>	Runs on CPU/modest RAM without stalling
24-32GB GPU (or 32GB Mac)	<code>qwen3-coder:30b</code>	Best coding performance per GB of VRAM
High-end GPU / workstation (40GB+)	<code>llama3.3:70b</code>	Best general-purpose quality locally
Need embeddings for RAG	<code>nomic-embed-text</code>	Small, fast, purpose-built for retrieval
Need vision/multimodal	<code>gemma4:12b</code>	Natively multimodal at manageable size

Always run `ollama show <model>` before committing to a large download it tells you the exact context window, default quantization,



AI Coach John

FOLLOW



PROITBRIDGE
Drive For Better Future

Repost

Calling your local model **from any app**

Ollama's background service exposes a REST API on **localhost** meaning any language or tool that can make an HTTP request can use your local models.

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Explain RAG in one sentence.",
  "stream": false
}'
```

💡 THE ADVANTAGE

This is exactly the same shape as calling a cloud LLM API (OpenAI, Anthropic) the difference is your request never leaves your machine, and there's no API key or billing involved.



Building it into **your actual project**

The official **ollama** Python package wraps the API into simple calls and it also works directly inside LangChain/LangGraph.

```
import ollama

response = ollama.chat(
    model="llama3.2",
    messages=[{"role": "user", "content": "Explain RAG."}]
)
print(response["message"]["content"])

# LangChain integration
from langchain_ollama import ChatOllama

llm = ChatOllama(model="llama3.2")
answer = llm.invoke("Explain RAG.")
```

Because the interface is so similar to cloud LLM SDKs, you can build & test an entire RAG pipeline locally with Ollama for free, then swap in a cloud model later for production with minimal code changes.



AI Coach John

FOLLOW



PROTIBRIDGE
Drive For Better Future

 **Repost**



Customizing a Model **with a Modelfile**

A **Modelfile** lets you create a variant of a model with a fixed system prompt or custom defaults baked in without writing code.

```
# Modelfile
FROM llama3.2

PARAMETER temperature 0.3
SYSTEM """You are a concise coding tutor. Always explain
with a short code example."""

# In terminal:
$ ollama create coding-tutor -f Modelfile
$ ollama run coding-tutor
```

This is a useful project idea: build a small library of custom Modelfiles for different personas (a tutor, a code reviewer, a terse assistant), and switch between them instantly.



They're not **mutually exclusive**

Use Ollama when:

Learning or building a portfolio project without API costs.

Privacy/data sensitivity is a hard requirement.

You want to experiment freely without rate limits.

Use a cloud API when:

You need the absolute best model quality/reasoning.

Your hardware can't run a model large enough for your needs.

You need to scale to many concurrent users.

Common pattern: build & test entirely on Ollama locally, then swap to a cloud model for final production deployment.



AI Coach John

FOLLOW



PROTIBRIDGE
Drive For Better Future

 Repost

COMMON PITFALLS

Mistakes beginners **make with** Ollama



Ignoring quantization

Ollama defaults to 4-bit; consider a higher-precision tag if you need maximum accuracy for sensitive tasks.



Not checking the context window

Feeding a long document into a model with a small context window causes it to silently lose earlier content.



Picking a model bigger than your hardware

Always check `ollama show`` and the VRAM/RAM guidance before a large download.



Forgetting to keep Ollama updated

New versions bring performance improvements and access to newly released models.



AI Coach John

FOLLOW



PROITBRIDGE
Drive For Better Future

 **Repost**